

OOP 練習 13：OO 事件——EVENTS / RAISE EVENT / SET HANDLER

學習目標

- 理解事件機制的三個角色：宣告（EVENTS）、發布（RAISE EVENT）、訂閱（handler 方法 + SET HANDLER）
- 會寫 handler 方法：FOR EVENT ... OF ...、IMPORTING 只接需要的參數、sender
- 會用 SET HANDLER ... FOR 物件 訂閱、ACTIVATION space 退訂，理解多訂閱者互不相識
- 會訂閱標準 API 的事件：cl_salv_events_table 的 double_click（op11 的 SALV 加上互動）
- 能說出事件跟「直接呼叫方法」的差別：解耦——這就是 Observer 模式（Design Pattern 課的前哨）

事前準備

ADT 建立全域類別 ZCL_0013_<你的姓名縮寫>_MON 與程式 ZR_0013_<你的姓名縮寫>，套件 \$TMP；SFLIGHT 要有資料（SAPBC_DATA_GENERATOR）。

觀念引入：誰呼叫誰？

到 op12 為止，互動都是「呼叫端認識被呼叫端」：報表 NEW 一個類別、呼叫它的方法。事件把方向反過來：發布者只宣布「發生了什麼事」，完全不知道（也不在乎）誰在聽；訂閱者自己掛上來。加一個新反應（寄信、記 log、發告警）只要多一個訂閱者，發布者一行都不用改——這就是解耦。

其實你早就用過事件：基礎課 ex10 的 AT LINE-SELECTION 就是「系統發事件、你寫 handler」；op11 的 SALV 工具列按鈕也是。本課補上「自己宣告、自己發布」的完整拼圖。

題目需求

情境：航班乘載率監控。掃描航班，乘載率低於門檻就發出 seats_low 事件；誰要處理（畫面告警、統計、寄信）由訂閱者自己決定。

1. 發布者（全域類別，答案 ZCL_0013_FLIGHT_MONITOR）：
 - TYPES tt_flights TYPE STANDARD TABLE OF sflight WITH DEFAULT KEY.
 - 宣告事件（EXPORTING 參數強制 VALUE(...) 傳值）：EVENTS seats_low EXPORTING VALUE(iv_carrid)... VALUE(iv_connid)... VALUE(iv_fldate)... VALUE(iv_pct) TYPE i.
 - constructor IMPORTING iv_threshold TYPE i DEFAULT 50，存進 READ-ONLY 屬性（op03 複習）
 - scan(it_flights)：逐筆算乘載率（seatsocc * 100 / seatsmax，跳過 seatsmax = 0），低於門檻就 RAISE EVENT seats_low EXPORTING ...。類別裡不准出現任何 WRITE——發布者不管誰在聽
2. 訂閱者一 lcl_alerter（demo 程式的 local class）：handler 方法 on_seats_low FOR EVENT seats_low OF zcl_oo13_flight_monitor IMPORTING iv_carrid iv_connid iv_fldate iv_pct sender.，WRITE 一行告警

- 3. 訂閱者二 `lcl_logger`：同一事件的另一個 handler，但 `IMPORTING` 只接 `iv_carrid`（參數可以只接用得到的），內容只做 `mv_count + 1`（`READ-ONLY` 屬性）
- 4. 主流程：SELECT 航班（UP TO 200 ROWS）→ NEW 發布者（門檻 50）與兩個訂閱者 → 兩句 `SET HANDLER ... FOR go_monitor` → `scan()` → 輸出 logger 統計
- 5. 退訂實驗：SET HANDLER go_alerter->on_seats_low FOR go_monitor ACTIVATION space. → 再 `scan()` 一次——告警不再輸出，logger 照數（累計）
- 6. 實戰——標準 API 事件：用 `cl_salv_table` 顯示航班（op11 技能），`go_alv->get_event()` 拿事件物件，local class handler `on_double_click FOR EVENT double_click OF cl_salv_events_table IMPORTING row column.`，SET HANDLER 註冊；雙擊某列 → MESSAGE ... TYPE 'I' 彈出該筆「已售 / 總座位」明細
- 7. 觀察並在註解寫下：發布者類別裡沒有任何輸出邏輯；兩個訂閱者互相不知道對方存在

預期結果

```
=== 第一次掃描：兩個訂閱者都在 ===
【告警】 AA    0017 2026/11/23 乘載率          31 %
【告警】 AZ    0555 2026/12/06 乘載率          42 %
( ...每筆低乘載航班一行... )
logger 統計：          18 筆低乘載告警
-----
=== 第二次掃描：alerter 已退訂，只剩 logger 在數 ===
logger 統計：          36 筆（兩次累計）
```

之後出現 SALV 畫面；雙擊任一列彈出 I 訊息（如 `AA 0017 2026/11/23`：已售 120 / 380 座）。

對比：直接呼叫 vs 事件

	直接呼叫方法	事件（EVENTS/SET HANDLER）
誰認識誰	呼叫端認識被呼叫端	發布者不認識訂閱者
加一個新反應	改呼叫端程式碼	加一個訂閱者，發布者不動
執行者數量	一次一個	0 到多個 handler 依註冊順序執行
沒人接會怎樣	不存在的方法 = 編譯錯誤	RAISE EVENT 安靜地什麼都不發生
對照	PERFORM / method call	ex10 report event、SALV/GUI 控制項事件

團隊實務備註

- 實務上「訂閱標準 API 事件」遠多於「自己發布」：`cl_gui_alv_grid` 的 `double_click / toolbar / user_command`、`cl_salv_events_table`、Workflow 事件都是同一套 SET HANDLER 機制——本課的訂閱技能直接可搬
- SET HANDLER ... FOR ALL INSTANCES：訂閱該類別所有物件的事件；CLASS-EVENTS（靜態事件）則 SET HANDLER 不帶 FOR
- handler 是同步執行的：RAISE EVENT 當下逐一跑完所有 handler 才回到發布者——別在 handler 裡做重活

- 註冊會讓發布者**持有訂閱者的參考**（訂閱者不會被垃圾回收）：長生命週期的程式（常駐畫面）記得在適當時機 **ACTIVATION space** 退訂
- 這個機制的設計思想就是 **Observer 模式**——Design Pattern 課會回來替它命名

思考題

1. 把兩句 SET HANDLER 都註解掉再跑 **scan()**——會發生什麼？跟 **PERFORM 不存在的form** 的下場差在哪？這個「安靜」是優點還是風險？
2. 事件的 EXPORTING 參數為什麼**強制傳值 (VALUE)**？如果允許傳參考，某個 handler 改了參數會發生什麼事？
3. ex10 的 **AT LINE-SELECTION + HIDE** 和本課的 **double_click handler + READ TABLE ... INDEX row**，各自怎麼知道「使用者點的是哪一筆」？哪個機制比較不容易踩殘留值的坑？

答案

見 **zcl_oo13_flight_monitor.clas.abap** (SAP 端類別 **ZCL_0013_FLIGHT_MONITOR**) 與 **zr_oo13_event_demo.prog.abap** (SAP 端程式 **ZR_0013_EVENT_DEMO**) 。